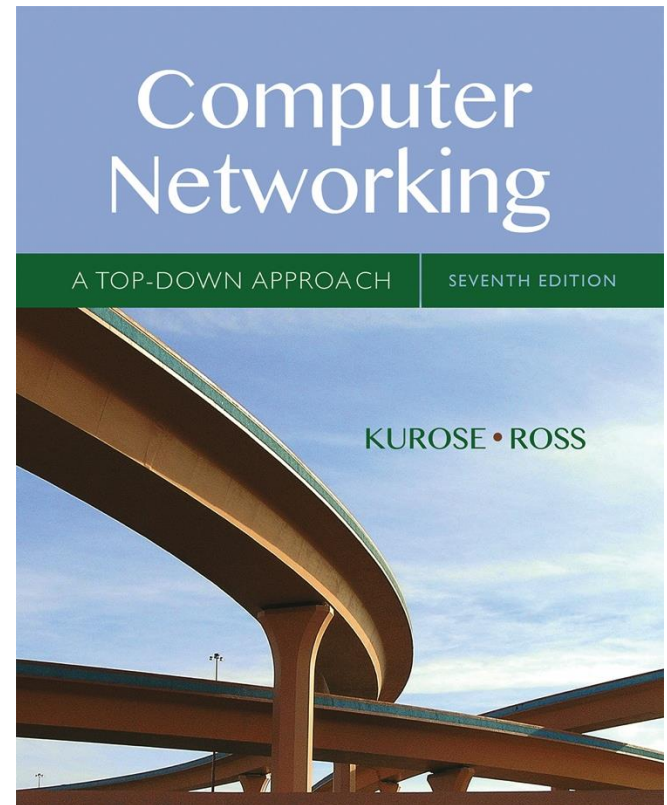


Chapter 9 Multimedia Networking



Computer Networking: A Top Down Approach

7th edition

Jim Kurose, Keith Ross

Pearson/Addison Wesley

April 2016

Multimedia networking: outline

9.1 multimedia networking applications

9.2 streaming *stored* video

9.3 voice-over-IP

9.4 protocols for *real-time* conversational applications

9.5 network support for multimedia

Voice-over-IP (VoIP)

- *VoIP end-end-delay one-way requirement*: needed to maintain “conversational” aspect
 - higher delays noticeable, impair interactivity
 - < 150 msec: good
 - > 400 msec bad
 - includes application-level (packetization, playout), network delays
- *session initialization*: how does callee advertise IP address, port number, encoding algorithms?
- *value-added services*: call forwarding, screening, recording
- *emergency services*: 911

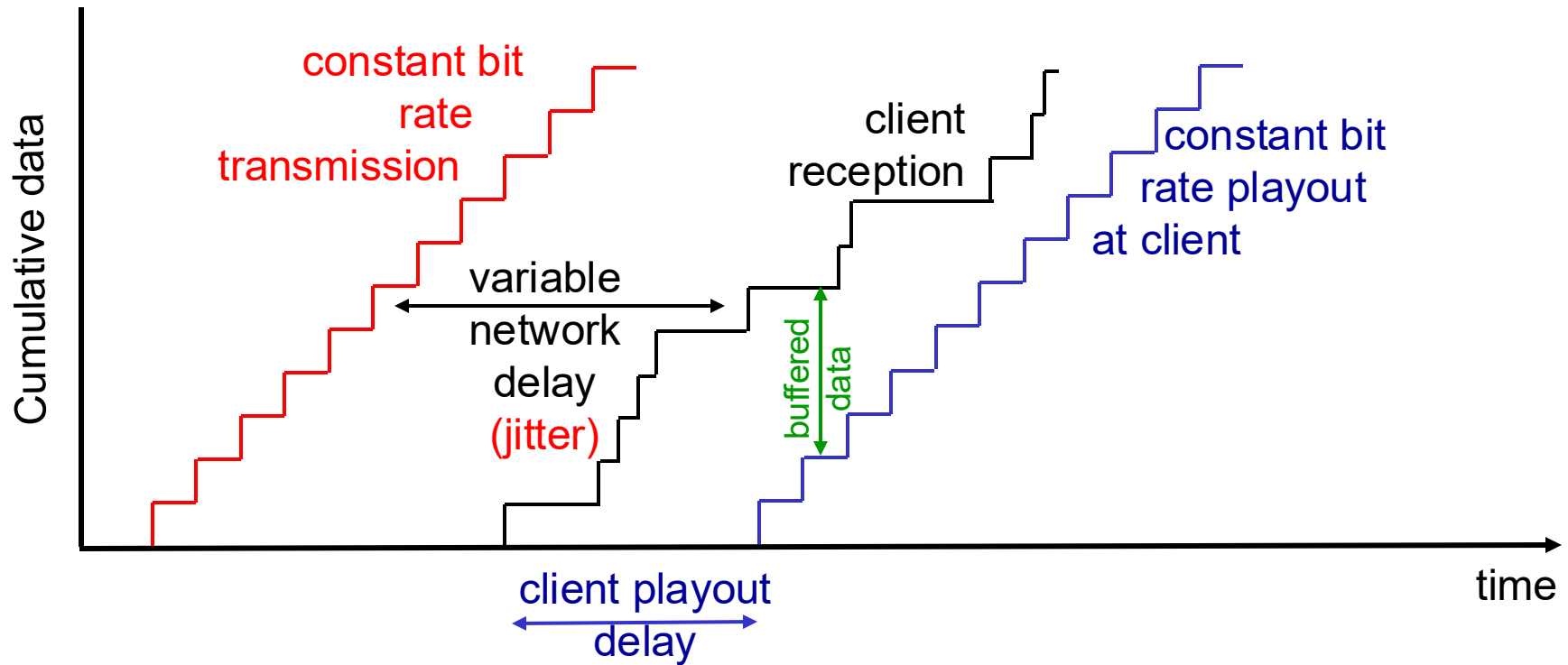
VoIP characteristics

- speaker's audio: alternating talk spurts, silent periods.
 - 64 kbps (=8Kbytes/sec) during talk spurt
 - pkts generated only during talk spurts
 - 20 msec chunks at 8 Kbytes/sec: 160 bytes of data
- application-layer header added to each chunk
- chunk+header encapsulated into UDP or TCP segment
- application sends segment into socket every 20 msec during talkspurt

VoIP: packet loss, delay

- *network loss*: IP datagram lost due to network congestion (router buffer overflow)
- *delay loss*: IP datagram arrives too late for playout at receiver
 - delays: processing, queueing in network; end-system (sender, receiver) delays
 - typical maximum tolerable delay: 400 ms
- *loss tolerance*: depending on voice encoding, loss concealment, packet loss rates between 1% and 10% can be tolerated

Delay jitter



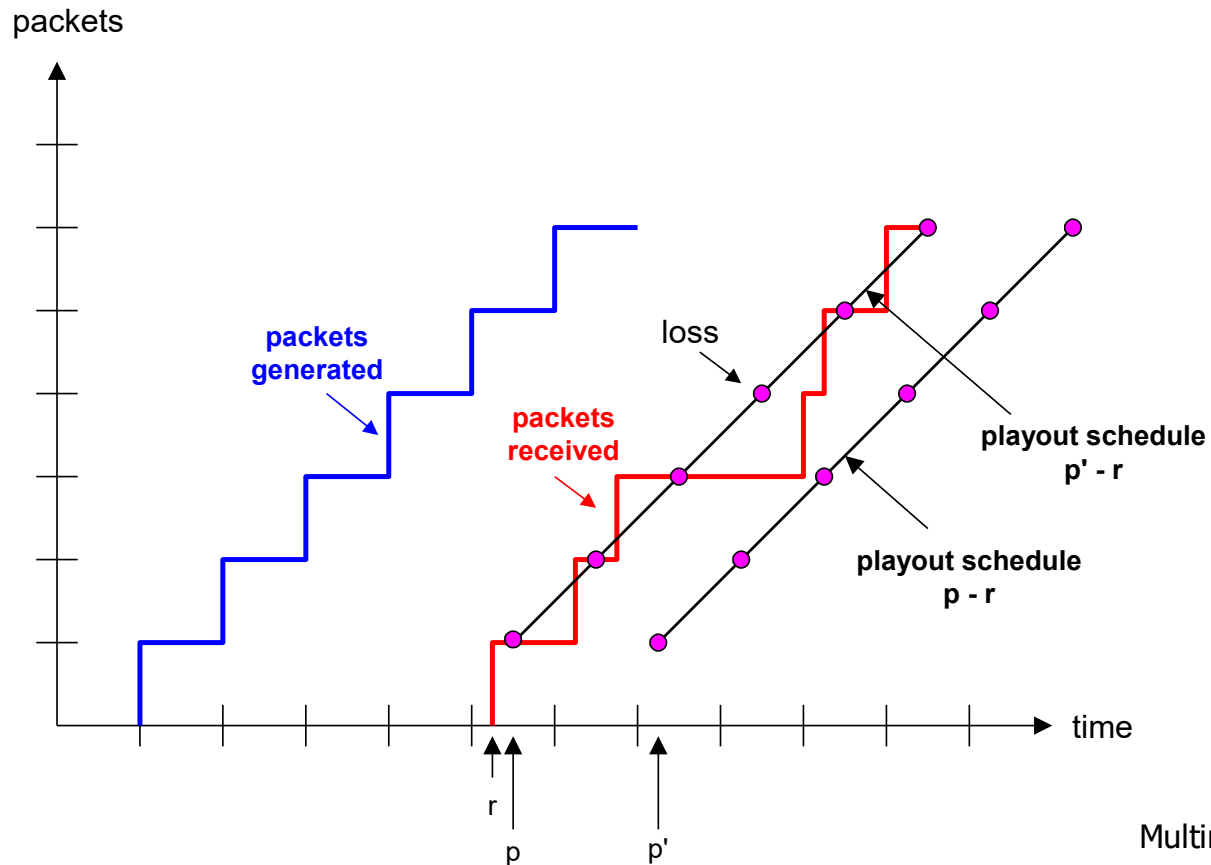
- Mye likt med streaming video...

VoIP: fixed playout delay

- receiver attempts to playout each chunk exactly q msecs after chunk was generated.
 - chunk has time stamp t : play out chunk at $t+q$
 - chunk arrives after $t+q$: data arrives too late for playout: data “lost”
- tradeoff in choosing q :
 - *large q : less packet loss*
 - *small q : better interactive experience*

VoIP: fixed playout delay

- sender generates packets every 20 msec during talk spurt.
- first packet received at time r
- first playout schedule: begins at p
- second playout schedule: begins at p'



Adaptive playout delay (I)

- **goal:** low playout delay, low late loss rate
- **approach:** adaptive playout delay adjustment:
 - estimate network delay, adjust playout delay at beginning of each talk spurt
 - silent periods compressed and elongated
 - chunks still played out every 20 msec during talk spurt
- adaptively estimate packet delay: (EWMA - exponentially weighted moving average, recall TCP RTT estimate):

$$d_i = (1-\alpha)d_{i-1} + \alpha (r_i - t_i)$$

<i>delay estimate after ith packet</i>	<i>small constant, e.g. 0.1</i>	<i>time received - time sent (timestamp) measured delay of ith packet</i>
--	-------------------------------------	---

Adaptive playout delay (3)

Q: How does receiver determine whether packet is first in a talkspurt?

- if no loss, receiver looks at successive timestamps
 - difference of successive stamps > 20 msec $\rightarrow$ talk spurt begins.
- with loss possible, receiver must look at both time stamps and sequence numbers
 - difference of successive stamps > 20 msec *and* sequence numbers without gaps $\rightarrow$ talk spurt begins.

VoiP: recovery from packet loss (I)

Challenge: recover from packet loss given small tolerable delay between original transmission and playout

- each ACK/NAK takes $\sim$ one RTT
- alternative: **Forward Error Correction (FEC)**
 - send enough bits to allow recovery without retransmission (recall two-dimensional parity in Ch. 5)

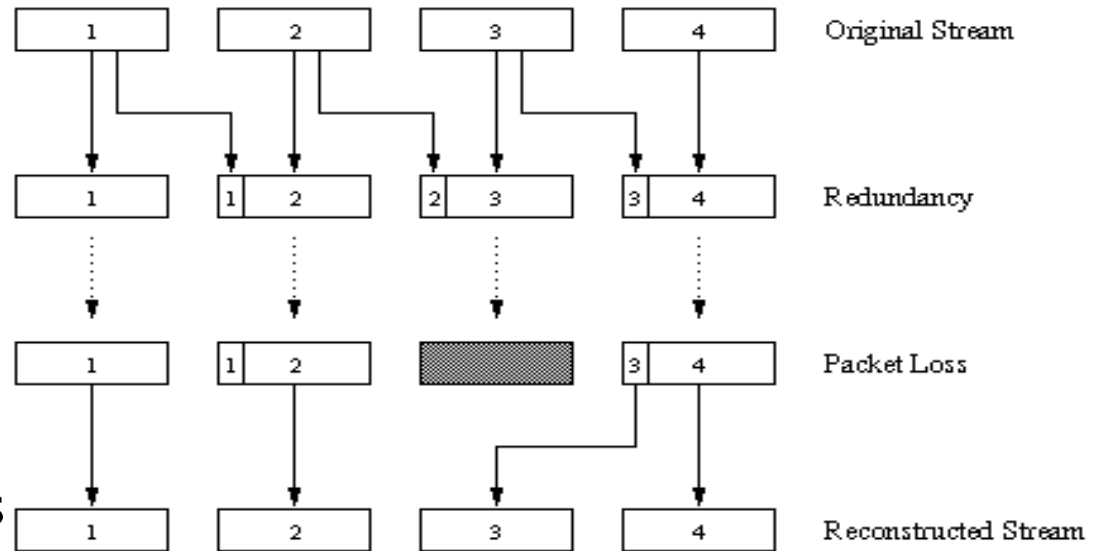
simple FEC

- for every group of n chunks, create redundant chunk by exclusive OR-ing n original chunks
- send $n+1$ chunks, increasing bandwidth by factor $1/n$
- can reconstruct original n chunks if at most one lost chunk from $n+1$ chunks, with playout delay

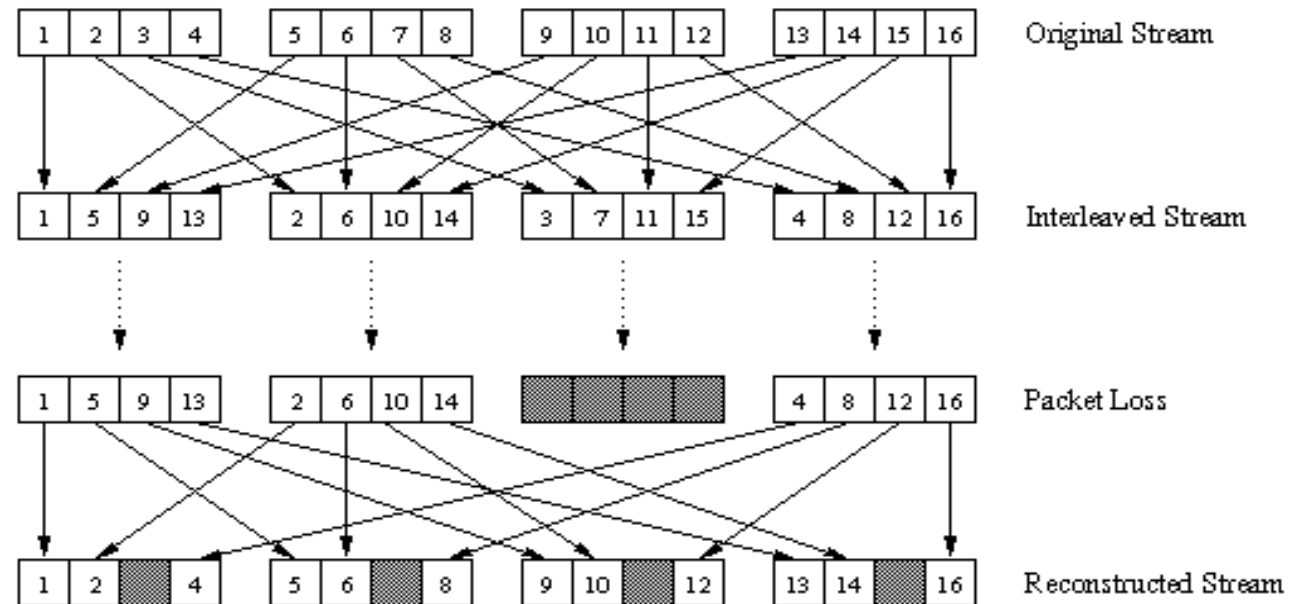
VoiP: recovery from packet loss (2)

another FEC scheme:

- “piggyback lower quality stream”
- send lower resolution audio stream as redundant information
- e.g., nominal stream PCM at 64 kbps and redundant stream GSM at 13 kbps
- non-consecutive loss: receiver can conceal loss
- generalization: can also append (n-1)st and (n-2)nd low-bit rate chunk



VoiP: recovery from packet loss (3)

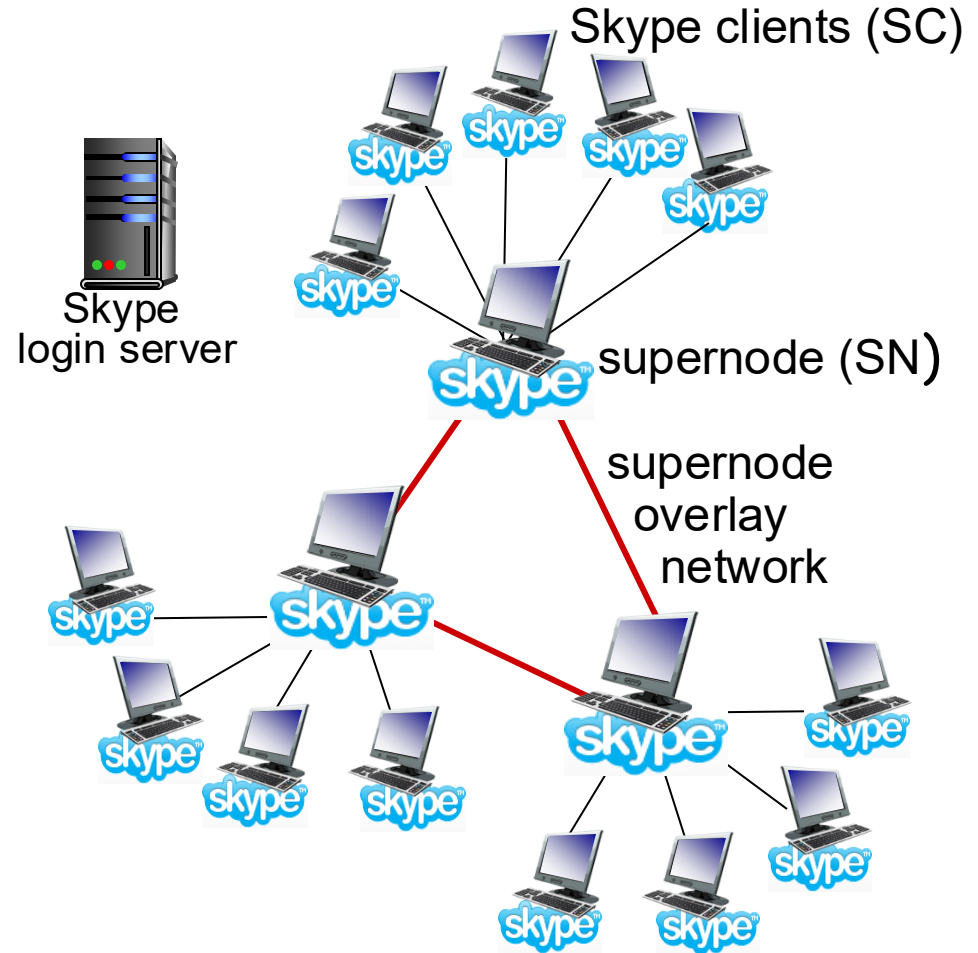


interleaving to conceal loss:

- audio chunks divided into smaller units, e.g. four 5 msec units per 20 msec audio chunk
- packet contains small units from different chunks
- if packet lost, still have *most* of every original chunk
- no redundancy overhead, but increases playout delay

Voice-over-IP: Skype

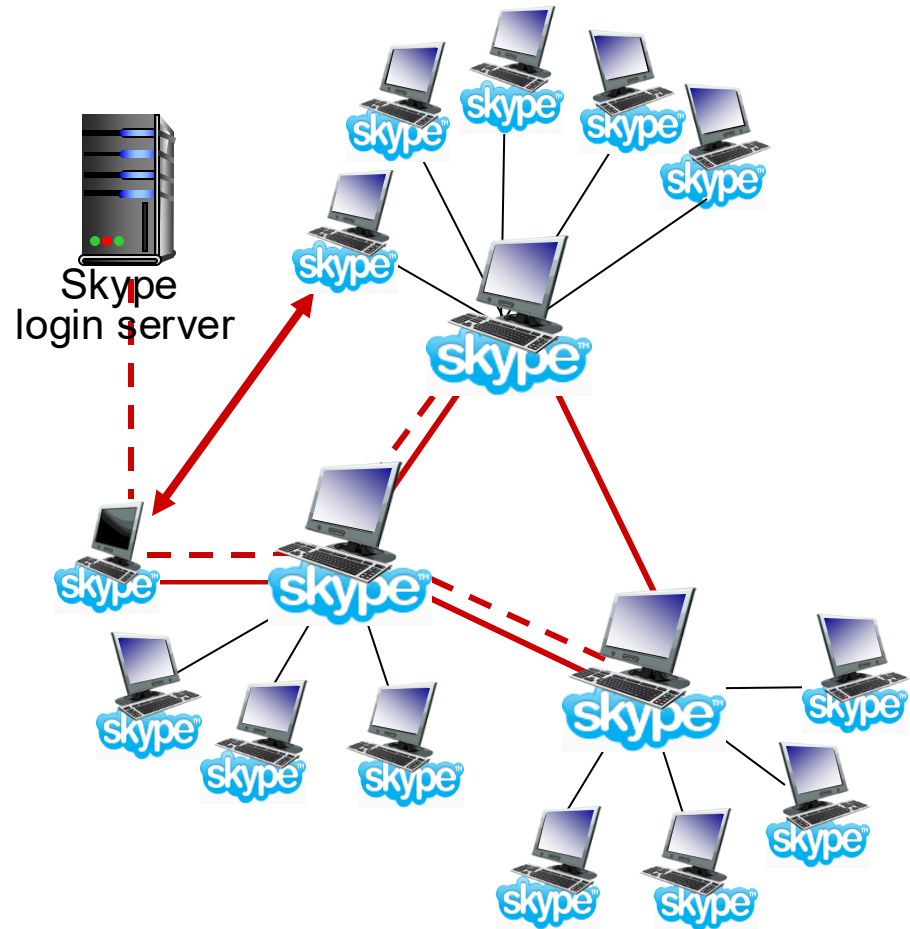
- proprietary application-layer protocol (inferred via reverse engineering)
 - encrypted msgs
- **P2P** components:
 - **clients**: Skype peers connect directly to each other for VoIP call
 - **super nodes (SN)**: Skype peers with special functions
 - **overlay network**: among SNs to locate SCs
 - **login server**



P2P voice-over-IP: Skype

Skype client operation:

1. joins Skype network by contacting SN (IP address cached) using TCP
2. logs-in (username, password) to centralized Skype login server
3. obtains IP address for callee from SN, SN overlay
 - or client buddy list
4. initiate call directly to callee



Skype: peers as relays

- **problem:** both Alice, Bob are behind “NATs”
 - NAT prevents outside peer from initiating connection to insider peer
 - inside peer *can* initiate connection to outside
- **relay solution:** Alice, Bob maintain open connection to their SNs
 - Alice signals her SN to connect to Bob
 - Alice’s SN connects to Bob’s SN
 - Bob’s SN connects to Bob over open connection Bob initially initiated to his SN

